

RAG- Retrieval-Augmented Generation

What is RAG?

- **Definition:** Retrieval-Augmented Generation (RAG) is a technique where a language model retrieves relevant documents or data from external sources before generating responses.
- **Purpose:** It addresses limitations of LLMs, such as outdated knowledge, hallucinations, and lack of domain-specific information.
- **Use Cases:** Question answering, customer support, legal/medical AI assistants, and enterprise knowledge systems.

Types of RAG (8 Variants)

Type	Description
1. Retriever-Centric	Focuses on optimizing retrieval quality (e.g., dense vector search).
2. Generator-Centric	Emphasizes generation control, ensuring retrieved data is faithfully used.
3. Hybrid Models	Combine retriever and generator optimization for balance.
4. Robustness-Oriented	Designed to handle noisy or adversarial inputs.
5. Multi-Hop RAG	Retrieves multiple linked documents for complex reasoning.
6. Federated RAG	Uses distributed retrieval sources across networks.
7. Adaptive RAG	Dynamically adjusts retrieval strategies based on query type.
8. Privacy-Preserving RAG	Ensures sensitive data is protected during retrieval.

Pros and Cons of RAG

Pros

- **Improved Accuracy:** Reduces hallucinations by grounding responses in external evidence.
- **Domain Flexibility:** Can adapt to specialized fields (law, medicine, finance).
- **Up-to-Date Knowledge:** Retrieves current information beyond model training cut-off.
- **Efficiency:** Avoids retraining large models for new knowledge.

Cons

- **Retrieval Quality Dependence:** Poor retrieval leads to weak answers.
- **Pipeline Complexity:** Requires orchestration between retriever and generator.

- **Latency Issues:** Real-time retrieval can slow response generation.
- **Robustness Challenges:** Vulnerable to noisy or adversarial inputs.

Architecture of RAG

The RAG pipeline typically consists of **four main components**:

1. **Query Encoder**

- Converts user input into embeddings (vector representations).
- Example: BERT or sentence transformers.

2. **Retriever**

- Searches external knowledge bases (databases, documents, APIs).
- Methods: Dense retrieval (vector similarity), sparse retrieval (BM25).

3. **Generator (LLM)**

- Produces responses conditioned on retrieved evidence.
- Example: GPT-style models integrated with retrieved context.

4. **Fusion Mechanism**

- Combines retrieved data with model knowledge.
- Approaches: Late fusion (post-retrieval), early fusion (integrated during decoding).

Workflow: User Query → Query Encoder → Retriever → Retrieved Documents → Generator → Final Response.